

Tools in LangChain

1. Recap of Journey So Far

- Playlist so far = **15 videos**.
 - **Part 1** – Fundamentals of LangChain (models, prompts, chains, etc.).
 - **Part 2** – Building RAG systems (document loaders, text splitters, vector stores, retrievers, RAG pipeline).
 - **Part 3** – Starting now → **Agents**.
 - Topics to cover:
 1. **Tools** (this video)
 2. **Tool Calling**
 3. **Agents (full system)**
-

2. Why Tools? (Conceptual Foundation)

♦ Core Abilities of LLMs

- **Think** (reasoning)
- **Speak** (generate language)

⚠ But LLMs **cannot act**:

- Cannot fetch live weather, book tickets, send tweets, run code, or query databases reliably.
 - Analogy: *LLM = a human with a brain & mouth but no hands/legs.*
-

♦ What Tools Do

- Tools = give LLM **hands & legs** to act.
- Technically: **a Python function packaged so LLM can understand & call it.**
- LLM decides **when to use which tool**.

Example:

- Tool = IRCTC train booking function.
- User: “Book me a train from Delhi to Mumbai.”
- LLM → invokes tool → books train → returns confirmation.

👉 More tools → more tasks LLM can perform.

3. Tools & Agents Connection

• **Agent Definition:**

An AI agent is an LLM-powered system that can autonomously *think, decide, and take actions* using external tools/APIs to achieve a goal.

- **Marriage of LLM (reasoning) + Tools (actions) = Agent.**
 - So tools are the **backbone of agentic systems** in LangChain & LangGraph.
-

4. Types of Tools in LangChain

♦ Built-in Tools

- Provided by LangChain team.
- Production-ready, minimal/no setup.
- Covers **popular use cases**:
 - DuckDuckGo Search → real-time web search.
 - Wikipedia Query Run → fetch + summarize Wikipedia data.
 - Python REPL → execute Python code safely.
 - Shell Tool → run terminal commands.

- RequestsGet → make HTTP requests.
- GmailSendMessage, Slack Tool, SQLDatabaseQuery → productivity & DB tasks.

👉 You just **import & use**, no need to write logic.

◆ Custom Tools

Needed when:

- Calling your **own APIs** (e.g., MakeMyTrip booking system).
- Encapsulating **business logic** unique to your app.
- Interacting with **databases, products, or apps**.

👉 Must be created when built-in tools don't fit.

5. How to Create Custom Tools

✅ Method 1: @tool Decorator (Simple)

Steps:

1. Write Python function with docstring.
2. Add type hints (int, str, etc.).
3. Decorate with @tool.

```
from langchain_core.tools import tool
```

```
@tool
```

```
def multiply(a: int, b: int) -> int:
```

```
    """Multiply two numbers."""
```

```
    return a * b
```

```
# Use tool
```

```
result = multiply.invoke({"a": 3, "b": 5})
```

```
print(result) # 15
```

- Tool auto-exposes:
 - .name
 - .description (from docstring)
 - .args (from type hints)
 - What LLM sees = **JSON Schema**, not code.
-

✅ Method 2: Structured Tool + Pydantic

- Stricter input validation.
- Use Pydantic models to enforce schema.

```
from langchain.tools import StructuredTool
```

```
from pydantic import BaseModel, Field
```

```
class MultiplyInput(BaseModel):
```

```
    a: int = Field(..., description="First number")
```

```
    b: int = Field(..., description="Second number")
```

```
def multiply(a: int, b: int) -> int:
```

```
    return a * b
```

```
multiply_tool = StructuredTool.from_function(  
    func=multiply,
```

```

name="Multiply",
description="Multiply two integers",
args_schema=MultiplyInput
)

```

👉 Useful for **production-grade systems**.

✅ Method 3: Inherit BaseTool Class

- Deepest level of customization.
- Extend BaseTool, define:
 - name
 - description
 - args_schema (with Pydantic)
 - _run() method (logic)

```

from langchain.tools import BaseTool
from pydantic import BaseModel, Field

```

```

class MultiplyInput(BaseModel):
    a: int = Field(..., description="First number")
    b: int = Field(..., description="Second number")

```

```

class MultiplyTool(BaseTool):
    name = "Multiply"
    description = "Multiply two integers"
    args_schema = MultiplyInput

```

```

def _run(self, a: int, b: int) -> int:
    return a * b

```

👉 Supports **async execution & concurrency**.

👉 Best for **enterprise apps**.

6. Toolkits in LangChain

- **Definition:** A toolkit = *collection of related tools* grouped for reusability.
- Example:
 - Google Drive Toolkit: upload, search, read files.
 - Math Toolkit: add, multiply functions.

Code Outline:

```

class MathToolkit:
    def get_tools(self):
        return [add_tool, multiply_tool]

```

👉 Improves **organization + reusability**.

7. Interview-Style Q&A

Q1. What problem do tools solve in LangChain?

👉 They give LLMs the ability to take actions (act as hands & legs).

Q2. How are tools related to agents?

👉 Agent = LLM (thinking + reasoning) + Tools (action execution).

Q3. Difference between built-in and custom tools?

- 👉 Built-in: ready-to-use (DuckDuckGo, Wikipedia, REPL).
- 👉 Custom: for app-specific APIs/business logic.

Q4. Name 3 ways to create custom tools.

- 👉 @tool decorator, StructuredTool (Pydantic), Inherit BaseTool.

Q5. Why use Pydantic for tools?

- 👉 Enforces strict input/output schema, safer for production.

Q6. What are toolkits and why use them?

- 👉 Collection of related tools, helps reuse across projects.
-

8. Real-World Applications

- **Travel Booking Agent** – IRCTC tool, hotel API tool, flight API tool.
 - **Research Assistant** – Web search + Wikipedia + summarizer.
 - **Developer Assistant** – Python REPL + shell tool + GitHub tool.
 - **Enterprise Apps** – Custom DB tools + API integrations.
-

✅ Summary:

- Tools are Python functions packaged for LLM use.
 - Built-in tools = plug & play, Custom tools = business-specific.
 - Three creation methods: @tool, StructuredTool, BaseTool.
 - Toolkits = reusable bundles.
 - Tools + LLM = **Agents**.
-